

Raport problem regresyjny

1. Opis badanego problemu

1.1 Charakterystyka zadania

Celem projektu jest stworzenie sieci neuronowej, która przewidzi końcowe ceny obrazów na aukcjach. Z perspektywy uczenia maszynowego jest to problem regresji – estymacja konkretnej wartości na podstawie cech fizycznych dzieła.

Rynek sztuki jest rynkiem specyficznym: charakteryzuje go silna asymetria informacji i duża zmienność cen. O ostatecznej wartości decydują nie tylko obiektywne parametry, takie jak wymiary, technika czy nazwisko twórcy, ale też aktualne trendy kolekcjonerskie. Tradycyjnie wyceną zajmują się rzeczoznawcy, co czyni ten proces kosztownym i subiektywnym. Nasz model ma na celu automatyzację tej pracy i wprowadzenie powtarzalnych standardów wyceny.

1.2 Zbiór danych

Nasz model opiera się na zbiorze danych `auction_results_color_svd.csv`, zawierającym historyczne wyniki aukcji sztuki. Każdy rekord to pojedynczy obraz, który opisaliśmy za pomocą dwóch grup cech:

Zmienne liczbowe (ciągłe):

- **TOTAL DIMENSIONS** – całkowite wymiary obrazu,
- **YEAR** – rok powstania dzieła,
- **Colorfulness Score** – komputerowy wskaźnik nasycenia i różnorodności barw,
- **SVD Entropy** – miara złożoności wizualnej na podstawie dekompozycji SVD.

Zmienne kategoryczne:

- **ARTIST** – autor dzieła,
- **TECHNIQUE** – technika wykonania (np. olej na płótnie, akwarela),
- **SIGNATURE** – obecność i rodzaj sygnatury,
- **CONDITION** – stan zachowania obrazu.

Aby model mógł przetworzyć zmienne kategoryczne, zastosowaliśmy kodowanie *One-Hot Encoding*, przekształcając je w rzadkie wektory binarne.

1.3 Zmienna docelowa i transformacja danych

Zmienną docelową modelu jest ostateczna cena sprzedaży w dolarach. Rozkład cen na rynku sztuki jest silnie prawoskośny – dominują prace stosunkowo tanie, a sporadyczne rekordy aukcyjne tworzą długi prawy ogon rozkładu. Przed treningiem sieci zlogarytmowaliśmy tę wartość, co rozwiązuje dwa praktyczne problemy: stabilizuje gradient podczas minimalizacji błędu (ekstremalnie drogie obrazy przestają dominować nad aktualizacją wag) i wymusza na sieci naukę rzędu wielkości ceny zamiast jej absolutnej wartości, co sprzyja generalizacji.

Dla wejściowych cech liczbowych przetestowaliśmy dwie strategie normalizacji – klasyczną standaryzację Z-Score oraz analogiczną transformację logarytmiczną.

2. Przegląd literatury

Prognozowanie cen na rynku sztuki jest trudne z fundamentalnego powodu: ostateczna wycena zależy od mieszaniny czynników obiektywnych i subiektywnych, z którymi tradycyjne modele statystyczne radzą sobie słabo. Głębokie sieci neuronowe rozwiązują problem nieliniowości, ale stwarzają nowy – przeuczenie. Mauer i Paszkiel (2024), autorzy zbioru danych użytego w tym projekcie, wykazali, że modele MLP bez silnej regularyzacji po prostu zapamiętują zaszumione dane aukcyjne zamiast wyciągać z nich ogólne wnioski. To ich obserwacja stała się główną motywacją do wdrożenia warstw Dropout w naszej architekturze.

Równie istotny jest dobór cech. Nho i Park (2019) pokazali, że modele (CNN) analizujące wyłącznie wygląd obrazu dają fatalne wyniki predykcyjne. Mauer i Paszkiel (2024) potwierdzili, że złożone metryki wizualne, takie jak entropia SVD, częściej wprowadzają szum niż użyteczną informację. Zgodnie z ich badaniami o ostatecznej wycenie decydują twarde atrybuty aukcyjne: nazwisko twórcy, wymiary płótna i historia wystaw.

Za punkt odniesienia dla naszych wyników przyjmujemy perceptron oparty wyłącznie na metadanych, który w badaniach Nho i Park (2019) osiągnął $R^2 = 0,4874$ na zlogarytmowanej cenie. Wynik ten wskazuje, że około 50% wyjaśnianej wariancji to realistyczny sufit predykcyjny dla modeli pozbawionych danych o nastrojach rynkowych i indywidualnych decyzjach kolekcjonerów.

3. Badanie Sztucznej Sieci Neuronowej

3.1 Metodologia

Do optymalizacji modelu wykorzystaliśmy metodę Grid Search, zmieniając każdorazowo jeden hiperparametr przy pozostałych stałych. Aby ograniczyć wpływ losowej inicjalizacji wag na wyniki, każdą konfigurację trenowaliśmy trzykrotnie i uśrednialiśmy rezultaty. Zbiór danych podzieliliśmy w standardowej proporcji 80/20 na część treningową i testową.

Odrzuciliśmy standardową metrykę MAPE (Mean Absolute Percentage Error) przy ocenie modeli, ponieważ zupełnie nie sprawdzała się przy specyfice rynku sztuki. Jej głównym problemem jest brak górnego ograniczenia i asymetryczne karanie modelu za przeszacowywanie tanich dzieł, co mocno zniekształcało nasze wyniki.

Wzór na MAPE:

$$MAPE = \frac{1}{n} \sum \frac{|F_t - A_t|}{A_t}$$

Zamiast tego użyliśmy metryki sMAPE (Symmetric MAPE):

$$sMAPE = \frac{1}{n} \sum \frac{2|F_t - A_t|}{|A_t| + |F_t|}$$

gdzie A_t to cena rzeczywista, a F_t to prognoza modelu. W odróżnieniu od klasycznego MAPE, ta miara jest symetryczna i zamyka wielkość błędu w stałym przedziale [0%-200%]. Dzięki temu mogliśmy sprawiedliwie oceniać i porównywać różne warianty naszej sieci.

3.2 Model Bazowy

Na etapie bazowym porównaliśmy obie strategie normalizacji, trenując oba warianty na prostej architekturze z domyślnymi parametrami.

Metryka	Transformacja Z-Score		Transformacja Logarytmiczna	
	Train	Test	Train	Test
MAE	104.60 \$	113.14 \$	104.74 \$	108.69 \$
MAPE	219.04 %	245.74 %	116.24 %	175.01 %
sMAPE	69.17 %	70.30 %	54.27 %	58.67 %
RMSE	275.5 \$	290.86 \$	348.70 \$	338.86 \$
R ²	0.6721	0.6175	0.4748	0.4809

Transformacja logarytmiczna wypadła wyraźnie lepiej według wszystkich głównych metryk: sMAPE na zbiorze testowym wyniosło 58,67% wobec 70,30% dla Z-Score, a MAE obniżyło się z 113,14 \$ do 108,69 \$. Jedyną metryką, w której Z-Score prowadzi, jest R² – 0,62 wobec 0,48. Nie jest to jednak zaskakujące: Z-Score nie kompresuje skrajnych wycen, więc sieć przypadkowo "trafia" w ogólny kształt rozkładu, ale popełnia przy tym znacznie większe błędy kwotowe. W kontekście praktycznej użyteczności modelu wycenowego to właśnie sMAPE i MAE są miarodajnymi miarami jakości predykcji. Wybieramy transformację logarytmiczną.

3.3 Badanie architektury sieci

Rozpoczęliśmy od optymalizacji pojemności dwóch warstw ukrytych, oceniając modele docelową miarą sMAPE.

Architektura	Śr. Czas [s]	Śr. sMAPE Train [%]	Śr. sMAPE Test [%]	Najlepsze sMAPE Test [%]
[32, 16]	2.48	56.90	60.67	58.71
[64, 32]	2.96	54.43	59.02	57.62
[128, 64]	4.77	54.29	59.41	57.95
[256, 128]	9.08	52.25	58.11	57.74
[512, 256]	19.79	54.62	60.34	57.12

Błąd sMAPE na zbiorze treningowym maleje monotonicznie wraz ze wzrostem liczby parametrów. Jednak z uwagi na zaszumienie danych, bardzo duże układy (np. [512, 256]) znacząco pogarszają średni wynik testowy (>60%) i wydłużają czas obliczeń, co sygnalizuje silne przeuczenie. Architektura [256, 128] oferuje najlepszy kompromis: posiada odpowiednią pojemność do wyłapywania rynkowych zależności, a jej wczesne objawy przeuczenia zostaną zniwelowane w kolejnych krokach technikami regularyzacji. Wybraliśmy układ [256, 128].

3.4 Badanie Współczynnika Uczenia (Learning Rate)

Następnie zbadaliśmy wpływ współczynnika uczenia (Learning Rate).

Learning Rate	Śr. Czas [s]	Śr. sMAPE Train [%]	Śr. sMAPE Test [%]	Najlepsze sMAPE Test [%]
0.100	9.46	53.96	59.60	58.45
0.050	9.09	55.39	59.21	57.62
0.010	9.44	56.20	58.38	58.33
0.001	8.83	70.84	70.62	70.54

Przetestowaliśmy cztery warianty współczynnika uczenia. LR = 0.001 okazał się zbyt małym krokiem – sieć nie zdążyła zbiec w założonej liczbie epok i zakończyła trening z błędem rzędu 70%. Na przeciwnym biegunie, LR = 0.100 bardzo sprawnie dopasował się do danych treningowych (53,96%), ale kosztem generalizacji – błąd testowy wyniósł 59,60%, co jest klasycznym objawem zbyt agresywnego kroku w końcowej fazie treningu. LR = 0.050 dał najlepszy pojedynczy wynik w jednej z prób (57,62%), lecz okazał się niestabilny między przebiegami. Najbardziej zrównoważone rezultaty zapewnił LR = 0.010 – najniższy średni błąd testowy (58,38%) przy jednocześnie najmniejszej wariancji wyników. Do dalszych eksperymentów wybraliśmy tę wartość.

3.5 Badanie rozmiaru paczki (Batch Size)

Kolejnym krokiem była optymalizacja rozmiaru paczki (Batch Size).

Batch Size	Śr. Czas [s]	Śr. sMAPE Train [%]	Śr. sMAPE Test [%]	Najlepsze sMAPE Test [%]
32	17.11	51.40	58.92	58.60
64	12.28	52.34	58.24	57.91
128	10.29	54.82	58.75	58.34
256	8.96	56.30	58.46	58.37

Wyniki pokazują dwie wyraźne tendencje. Po pierwsze, małe paczki (32) silnie dopasowują się do danych treningowych (51,40%), ale generalizują najgorzej – błąd testowy wynosi 58,92%. Wynika to ze stochastyczności gradientu: przy małym batch każda aktualizacja wag opiera się na bardzo niereprezentacyjnej próbce, co wprowadza szum utrudniający stabilną zbieżność. Po drugie, różnice między wariantami na zbiorze testowym są marginalne – zaledwie 0,7 punktu procentowego między najlepszym (58,24%) a najgorszym wynikiem (58,92%), co sugeruje, że architektura zbliżyła się do swojego limitu informacyjnego dla tych danych.

Batch Size = 64 dał najniższy średni błąd testowy (58,24%) i najlepszy pojedynczy przebieg (57,91%). Tę wartość wybieramy do dalszych eksperymentów.

3.6 Badanie Funkcji Aktywacji

Kolejnym krokiem był dobór funkcji aktywacji dla warstw ukrytych. Przetestowaliśmy cztery najpopularniejsze warianty.

Funkcja Aktywacji	Śr. Czas [s]	Śr. sMAPE Train [%]	Śr. sMAPE Test [%]	Najlepsze sMAPE Test [%]
ReLU	21.35	52.43	58.21	58.03
Leaky ReLU	29.53	52.44	58.26	58.20
Tanh	19.58	62.03	62.76	62.56
Sigmoid	21.40	68.19	68.03	68.00

Klasyczny Sigmoid oraz Tanh wyraźnie ustąpiły miejsca rodzinie funkcji ReLU, padając ofiarą problemu zanikającego gradientu (saturacja przy dużych wejściach) – co skutecznie blokowało uczenie nawet w tak płytkiej architekturze. Z kolei wariant Leaky ReLU dał wyniki bardzo zbliżone do klasycznego, nie dając wyraźnej przewagi predykcyjnej.

W docelowym modelu zostajemy przy klasycznym ReLU. Skutecznie zapobiega ono zanikaniu gradientu i to właśnie przy nim sieć osiągnęła najlepsze, najbardziej stabilne wyniki na obu zbiorach danych.

3.7 Badanie Liczby Epok

Liczba epok determinuje całkowity czas przetwarzania zbioru danych. Zbyt krótki trening prowadzi do niedouczenia (underfitting), zbyt długi – do przeuczenia (overfitting), gdy model zaczyna zapamiętywać szum zamiast uogólniać.

Liczba Epok	Śr. Czas [s]	Śr. sMAPE Train [%]	Śr. sMAPE Test [%]	Najlepsze sMAPE Test [%]
50	6.18	55.47	58.78	58.49
100	12.30	52.72	58.58	58.17
150	18.92	50.36	58.12	57.99
200	24.75	49.13	58.50	58.19
300	37.44	47.18	58.83	58.29

Trening przez 50 i 100 epok jest niewystarczający – sieć wciąż nie osiągnęła swojego potencjału. Z kolei przy 200 i 300 epokach pojawia się klasyczny symptom przeuczenia: błąd treningowy ciągle spada (nawet do 47,18%), ale błąd testowy zaczyna zauważalnie rosnąć (do 58,83%). Wariant 150 epok oferuje najlepszy bilans: osiąga absolutne minimum sMAPE testowego (58,12%) dokładnie w punkcie, zanim model zacznie tracić zdolność do generalizacji.

3.8 Badanie Współczynnika Dropout

Aby zniwelować przeuczenie, na które świadomie pozwoliliśmy w poprzednich etapach, wprowadziliśmy do architektury warstwę regularyzacyjną Dropout.

Wspt. Dropout	Śr. Czas [s]	Śr. sMAPE Train [%]	Śr. sMAPE Test [%]	Najlepsze sMAPE Test [%]
0.0	41.50	50.53	58.42	57.94
0.2	47.27	53.39	58.70	57.65
0.3	50.97	52.34	57.05	56.62
0.4	54.91	53.32	57.53	56.67
0.5	57.47	54.15	57.57	56.64
0.8	45.66	56.47	57.66	56.89

Bez regularyzacji (0.0) sieć klasycznie uczyła się na pamięć – błąd treningowy zmalał do 50,53%, ale na teście model wypadł najgorzej (58,42%). Z drugiej strony, przesadzenie z Dropoutem do 0.8 mocno spowolniło model. Model wpadał w *underfitting*, błąd na treningu jak i na testowym wzrósł.

Strzałem w dziesiątkę okazało się 0.3. Błąd treningowy w tym wariancie celowo jest trochę wyższy, bo sieć przestaje dopasowywać się do szumu, ale za to cały proces w końcu się ustabilizował.

Ostatecznie to właśnie Dropout na poziomie 0.3 wszedł do końcowej architektury. Dał nam najniższy średni błąd na teście (57,05%) i najlepszy pojedynczy strzał (56,62%), a jako bonus model trenował się odczuwalnie szybciej.

3.9 Badanie wielkości próby treningowej

Ostatnim badanym parametrem był podział danych na zbiór uczący i testowy. Przetestowaliśmy pięć wariantów proporcji.

Podział (Train/Test)	Śr. Czas [s]	Śr. sMAPE Train [%]	Śr. sMAPE Test [%]	Najlepsze sMAPE Test [%]
50%/50%	33.84	50.96	58.53	58.19
60%/40%	39.59	51.38	58.14	57.47
70%/30%	49.00	52.69	58.82	58.66
80%/20%	52.36	53.31	57.93	57.22
90%/10%	57.63	51.78	56.46	55.94

Wyniki nie układają się w prostą zależność – więcej danych treningowych nie zawsze przekłada się na lepszą generalizację. Wariant 70/30 wypadł gorzej niż 60/40 (58,82% vs 58,14%), co sugeruje, że przy tym podziale zbiór testowy stał się zbyt mały, żeby stabilnie

oceniać model. Dopiero przy 80/20 błąd testowy wyraźniej się poprawia (57,93%), utrzymując jednocześnie wystarczającą próbę do wiarygodnej ewaluacji.

Wariant 90/10 dał matematycznie najniższe wyniki (56,46%), ale celowo go odrzucamy – 10% próbek przy tak dużej zmienności cen na rynku sztuki to zbyt mała i niereprezentacyjna próba, by rzetelnie ocenić przeuczenie.

Do dalszych eksperymentów wybieramy podział 80/20 jako najlepszy kompromis między ilością danych treningowych a wiarygodnością oceny modelu.

3.10 Badanie wpływu Entropii SVD

Na koniec zweryfikowaliśmy hipotezę z literatury (Mauer i Paszkiel, 2024), że złożone cechy wizualne wprowadzają więcej szumu niż użytecznej informacji.

Wariant Danych	Śr. Czas [s]	Śr. sMAPE Train [%]	Śr. sMAPE Test [%]	Najlepsze sMAPE Test [%]
Z entropią SVD	52.34	53.04	57.81	56.64
Bez entropii SVD	52.12	52.93	57.25	57.15

Wyniki ją potwierdzają: wariant bez entropii SVD osiągnął niższy średni błąd testowy (57,25% vs 57,81%). Warto jednak odnotować, że najlepszy pojedynczy przebieg należał do wariantu z entropią (56,64% vs 57,15%) – cecha ta potrafi więc pomóc, ale tylko nieregularnie, co w praktyce czyni ją zawodną.

Z końcowego modelu usuwamy entropię SVD. Poprawa średniego błędu o 0,56 punktu procentowego przy jednoczesnym wzroście stabilności wyników to argument wystarczający – szczególnie że decyzja ta jest zgodna z obserwacjami literaturowymi dotyczącymi dominacji twardych metadanych aukcyjnych nad analizą wizualną.

3.11 Model końcowy

Dane: Transformacja logarytmiczna, bez Entropii SVD

Podział: Train (80%) / Test (20%)

Architektura: Wejście(414) -> ReLu(256) -> Drop(0.3) -> ReLu(128) -> Drop(0.3) -> L(1)

Trening: LR = 0.01, Batch = 64, Epoki = 150

Metryka	Model Bazowy		Model Końcowy	
	Train	Test	Train	Test
MAE	104.74 \$	108.69 \$	100.52 \$	107.42 \$
MAPE	116.24 %	175.01 %	121.70 %	174.46 %
sMAPE	54.27 %	58.67 %	53.34 %	57.61 %
RMSE	348.70 \$	338.86 \$	330.63 \$	327.46 \$
R ²	0.4748	0.4809	0.5274	0.5152

Zoptymalizowany model poprawił wyniki względem bazowego we wszystkich kluczowych metrykach. sMAPE na zbiorze testowym spadł z 58,67% do 57,61%, MAE obniżyło się o nieco ponad dolar (108,69 \$ → 107,42 \$), a RMSE zmniejszyło się o ponad 11 \$, co oznacza lepsze radzenie sobie z ekstremalnymi wycenami. Najbardziej odczuwalny jest wzrost R² – z 0,48 do 0,51 – model wyjaśnia już ponad połowę wariacji cen.

Warto jednak zachować umiarkowanie w interpretacji: poprawa jest realna, ale skromna. Fakt, że tak rygorystyczna optymalizacja przesunęła sMAPE tylko o około jeden punkt procentowy, potwierdza wcześniejszą obserwację o informacyjnym limicie tych danych – a nie słabość modelu. Zgodnie z wynikami Nho i Park (2019) (R² ≈ 0,49 dla modelu opartego wyłącznie na metadanych), osiągnięty poziom predykcji mieści się w górnych granicach tego, co jest realistycznie osiągalne bez danych o nastrojach rynkowych.

4. Badania dla UM

4.1 Matematyczne podstawy działania modeli

Regresja Ridge (L2)

Model ten szuka wektora wag, w który minimalizuje sumę kwadratów błędów (jak w zwykłej regresji liniowej), ale dodaje karę za zbyt duże wartości wag. Funkcja celu, którą model minimalizuje, wyraża się wzorem:

$$J(\mathbf{w}) = \sum_{i=1}^N (y_i - \mathbf{x}_i^T \mathbf{w})^2 + \alpha \sum_{j=1}^p w_j^2$$

Gdzie:

- alpha to hiperparametr sterujący siłą regularyzacji (im większy, tym silniejsza kara).
- Drugi człon równania zapobiega dopasowywaniu się modelu do szumu w danych (przeuczeniu).

Drzewo Decyzyjne (CART)

Algorytm rekurencyjnie dzieli przestrzeń cech na prostokątne regiony R_m . Predykcja dla nowej obserwacji wpadającej do regionu R_m to po prostu średnia wartość zmiennej objaśnianej z próbek uczących w tym regionie:

$$\hat{y}_{R_m} = \frac{1}{N_m} \sum_{x_i \in R_m} y_i$$

Podziałów dokonuje się zachłannie, szukając cechy j i punktu podziału s , które minimalizują błąd średniokwadratowy (MSE) w powstałych podzbiorach:

$$\min_{j,s} \left[\min_{c_1} \sum_{x_i \in R_1(j,s)} (y_i - c_1)^2 + \min_{c_2} \sum_{x_i \in R_2(j,s)} (y_i - c_2)^2 \right]$$

Las Losowy

Jest to rozszerzenie drzew decyzyjnych wykorzystujące metodę *baggingu* (Bootstrap Aggregating). Model trenuje B niezależnych drzew decyzyjnych na wylosowanych ze zwracaniem podzbiorach danych oraz przy użyciu losowych podzbiorów cech. Ostateczna predykcja dla próbki x jest średnią arytmetyczną z predykcji wszystkich pojedynczych drzew:

$$\hat{y}_{RF}(x) = \frac{1}{B} \sum_{b=1}^B \hat{y}_b(x)$$

Dzięki uśrednianiu, Las Losowy drastycznie redukuje wariancję modelu, rozwiązując główny problem pojedynczych drzew, jakim jest podatność na przeuczenie.

K-Najbliższych Sąsiadów (KNN)

Algorytm ten nie posiada jawnej fazy uczenia parametrycznego. Dla nowej obserwacji x oblicza odległość (najczęściej euklidesową) do wszystkich punktów w zbiorze treningowym:

$$d(x, x_i) = \sqrt{\sum_{j=1}^p (x_j - x_{ij})^2}$$

Następnie identyfikuje zbiór $N_K(x)$ zawierający K próbek o najmniejszej odległości. Predykcja to uśredniona wartość zmiennej celu z tego sąsiedztwa:

$$\hat{y}(x) = \frac{1}{K} \sum_{x_i \in N_K(x)} y_i$$

4.2 Tabele z wynikami

Poniższe tabele zestawiają wyniki wszystkich przeprowadzonych eksperymentów, pogrupowane według modeli i testowanych wartości parametrów. Analizę przeprowadziliśmy z podziałem na transformację Z-Score i Logarytmiczną rozkładu cen.

- Transformacja Z-Score

Model (parametr)	Wartość	R ²	sMAPE [%]	MAE [\$]	RMSE [%]	Czas [s]
Las Losowy (n_estimators)	10	0.6573	53.91	104.06	275.35	5.74
	50	0.6869	52.93	101.07	263.19	29.86
	100	0.6987	52.89	99.86	258.17	58.13
	200	0.7014	52.89	99.71	257.00	115.61
K-Najbliższych Sąsiadów (n_neighbors)	3	0.4598	58.85	124.08	345.70	0.0145
	5	0.4402	59.36	128.06	351.93	0.0038
	10	0.3746	62.56	139.06	371.96	0.0039
	20	0.3315	66.37	149.47	384.56	0.0038
Drzewo Decyzyjne (max_depth)	5	0.2726	79.19	176.44	401.15	0.14
	10	0.2932	72.49	154.47	395.43	0.24
	20	0.3296	65.95	136.33	385.11	0.41
	50	0.3481	60.88	122.63	379.76	0.74
Regresja Ridge (alpha)	0.1	0.3241	87.47	173.53	386.70	0.04
	1.0	0.3240	87.72	172.19	386.73	0.02
	10.0	0.3002	90.05	172.28	393.47	0.02
	100.0	0.2004	90.11	180.86	420.59	0.02

- Transformacja Logarytmiczna

Model (parametr)	Wartość	R ²	sMAPE [%]	MAE [\$]	RMSE [%]	Czas [s]
Las Losowy (n_estimators)	10	0.5808	56.43	99.81	304.52	3.13
	50	0.5753	54.81	97.51	306.54	16.22
	100	0.5808	54.80	97.07	304.53	32.51
	200	0.5824	54.65	96.65	303.96	65.24
K-Najbliższych Sąsiadów (n_neighbors)	3	0.3828	60.92	117.44	369.52	0.0027
	5	0.3840	60.09	116.44	369.17	0.0041
	10	0.3061	61.41	125.00	391.81	0.0038
	20	0.2062	63.12	133.53	419.06	0.0040
Drzewo Decyzyjne (max_depth)	5	0.1157	68.10	144.07	442.29	0.18
	10	0.1189	65.23	138.14	441.51	0.24
	20	0.3067	62.57	125.58	391.64	0.33
	50	0.4663	61.90	120.42	343.63	0.48
Regresja Ridge (alpha)	0.1	0.3216	61.35	124.24	387.41	0.07
	1.0	0.3109	61.18	124.28	390.46	0.03
	10.0	0.2074	61.59	130.20	418.73	0.03
	100.0	0.0746	65.81	144.64	452.46	0.03

4.3 Analiza wyników

- **Las Losowy (Random Forest):** Osiągnął najwyższą skuteczność predykcyjną ($R^2 \approx 0.70$, sMAPE = 52,89% przy Z-Score), co potwierdza silnie nieliniowy charakter wyceny sztuki. Algorytm osiąga optymalne wyniki przy 100 drzewach – dalsza rozbudowa jedynie podwaja czas treningu bez istotnego przyrostu jakości (wzrost R^2 o 0.003).
- **Regresja Ridge:** Konsekwentnie generowała najgorsze wyniki, dowodząc, że modele liniowe nie radzą sobie z danymi rozciągniętymi na kilka rzędów wielkości. Algorytm ten wyraźnie zyskuje jednak na transformacji logarytmicznej, która kompresuje skrajne wartości i pozwala obniżyć błąd sMAPE z 87% do 61%.
- **Algorytm KNN (k-najbliższych sąsiadów):** Działał najskuteczniej przy małych wartościach parametru K. Wynika z tego, że dzieła o zbliżonej wartości rynkowej

tworzą gęste skupiska w przestrzeni cech, a uśrednianie wyników po szerszym sąsiedztwie prowadzi do utraty kluczowych informacji.

- **Metody normalizacji (Z-Score vs Logarytm):** Skuteczność transformacji okazała się ściśle zależna od typu modelu. Las Losowy i KNN wymagały do optymalnego działania standaryzacji Z-Score, podczas gdy modele liniowe potrzebowały transformacji logarytmicznej, aby trafniej szacować wartość najtańszych obiektów.

5. Wnioski

Celem projektu było zbadanie, w jakim stopniu modele uczenia maszynowego i sztuczna sieć neuronowa zaimplementowana od podstaw są w stanie przewidywać ceny dzieł sztuki na podstawie ich atrybutów fizycznych i metadanych aukcyjnych.

Las Losowy osiągnął najwyższą skuteczność (sMAPE = 52,89%, $R^2 = 0,699$), wyraźnie przewyższając autorską sieć neuronową (sMAPE = 57,61%, $R^2 = 0,515$). Jako model nieparametryczny lepiej radzi sobie ze skośnym rozkładem cen i nieliniowymi interakcjami na stosunkowo małym zbiorze danych.

Zbudowany przez nas od podstaw model przekracza literaturowy punkt odniesienia $R^2 = 0,487$ (Nho i Park, 2019). Wynik $R^2 = 0,515$ dowodzi poprawnej architektury i mieści się w górnych granicach skuteczności dla modeli bazujących na metadanych.

Transformacja logarytmiczna zmiennej docelowej okazała się niezbędna niezależnie od klasy modelu, choć jej wpływ był silniejszy w przypadku SSN i regresji Ridge niż metod drzewiastych.

6. Bibliografia

1. Mauer, P., & Paszkiel, S. (2024). Tabular Data Models for Predicting Art Auction Results. *Applied Sciences*, 14(23), 116. <https://doi.org/10.3390/app142311006>
2. Nho, H., & Park, H. (2019). *The Art of predicting Art Auction Price*, Stanford University, https://cs230.stanford.edu/projects_fall_2019/reports/26261328